

Penetration Testing

AgriDex (DAO)

Table of Contents

Executive Summary	4
Project Context	4
Penetration Test scope	7
Security Rating	8
Code Quality	9
Penetration Test Resources	9
Dependencies	9
Severity Definitions	10
Penetration Test Findings	11
Centralisation	16
Conclusion	17
Our Methodology	18
Disclaimers	20
About Hashlock	21

CAUTION

THIS DOCUMENT IS A SECURITY PENETRATION TEST REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The AgriDex team partnered with Hashlock to conduct a security Penetration Test of their AgriDex Project code base. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed tools were secure.

Project Context

AgriDex is a Solana-based blockchain platform revolutionizing the global agricultural market by tokenizing real-world agricultural assets. It aims to enhance transparency, reduce transaction costs, and ensure instant settlement across the agricultural supply chain. Leveraging NFTs and a decentralized marketplace, AgriDex facilitates secure, near-instant transactions while lowering fraud risks. It supports global trade by providing transparent ESG credentials and compliance with sustainability regulations. The platform's \$AGRI token incentivizes users and enables decentralized governance through a DAO structure.

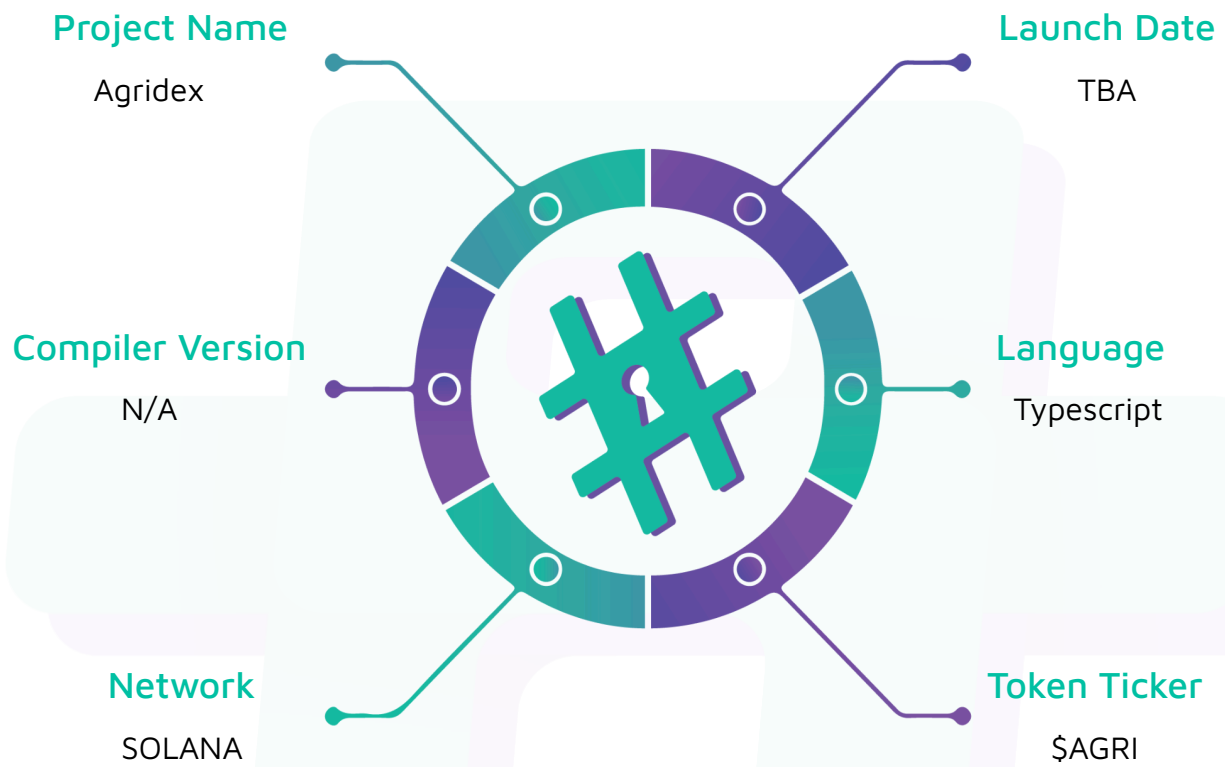
Project Name: Agridex

Compiler Version: N/A

Website: www.agridex.io

Logo:



Visualised Context:

Project Visuals:

AGRIDEX & SACRI TOKEN LAUNCHING Q4 2024

Bringing the \$2.7 Trillion Agricultural Market On-Chain

AgriDex is a Solana Backed RWA Marketplace
Reshaping the Global Agricultural Industry

[Learn about AgriDex](#)

[Download 2024 AGRIDEX WHITEPAPER](#)

BACKED BY SOLANA FOUNDATION


BUILT BY A TEAM FROM

 CITADEL

 Goldman Sachs

 SOLANA

 Deutsche Bank

 Google

 Trafigura

 DeepMind

NEW STRATEGIC PARTNERSHIP

SOLANA becomes AgriDex partner

We are partnering with Solana to bring the
\$2.7 Trillion Global Agriculture Industry on-chain

 SOLANA
FOUNDATION

Learn How AgriDex is Revolutionising the Agricultural
Industry

AGRIDEX'S JOURNEY

PRE-SALE LAUNCHING Q4 2024

The AgriDex Token

Token utility

AgriDex tokens will be used to incentivise and reward users based on trade volumes, platform engagement and community engagement.

Governance

Token holders can vote on elections of directors, providing the ability to help choose the new leaders of your company.

DAO

The AgriDex DAO will be formed and collectively managed and owned by its members involved in the DAO's token distribution.



AgriDex Governance Token

[Download Whitepaper](#)

\$0.10

Airdrop available soon

[Get airdrop notification](#)



Hashlock Pty Ltd

Penetration Test scope

At Hashlock, we executed a comprehensive penetration test on the Agridex project. Our scope included a detailed security assessment, where we rigorously examined the code to identify vulnerabilities and assess overall efficiency. The testing process involved a meticulous manual review, supplemented by advanced software-assisted techniques, to ensure thorough analysis and accuracy.

Only the folders cli/ and core/ were in the test scope, including all files they contain.

Description	AgriDex Project Code Base
Platform	Solana / Typescript
Penetration Test Date	October, 2024
GitHub Repository	https://github.com/AgriDex-International/agridex-treasury/tree/98cdbab5a24c57a5f9b453d7a9adff74cba2bb32/solana-tools-client/packages
GitHub Commit Hash	98cdbab5a24c57a5f9b453d7a9adff74cba2bb32

Security Rating

After Hashlock's Penetration Test, we found the code base to be **"Secure"**. The code follows simple logic, with correct and detailed ordering.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Penetration Test Findings](#) section.

We initially identified some significant vulnerabilities that have since been addressed.

Hashlock found:

1 High-severity vulnerabilities

1 Low-severity vulnerabilities

2 Gas Optimisations

Caution: *Hashlock's Penetration Tests do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Code Quality

This Penetration Test scope involves the code bases of the Agridex project, as outlined in the Penetration Test Scope section. All contracts, libraries, and interfaces mostly follow standard best practices to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Penetration Test Resources

We were given the Agridex projects code base code in the form of GitHub access.

As mentioned above, code parts are well-commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments help understand the overall architecture of the protocol.

Dependencies

Per our observation, the libraries used in this code base's infrastructure are based on well-known industry-standard open-source projects.

Apart from libraries, its functions are used in external code base calls.

Severity Definitions

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system's performance or security.

Penetration Test Findings

High

[H-01] Solana CLI tools - Insecure keypair/secret key storage

Description

It was determined that the tool stores a keypair in a plaintext file which is read during tool usage, for example in the below file:

`solana-tools-client/packages/cli/src/utls/read-keypair-from-file.ts`

```
import { readFileSync } from "node:fs";

import { resolve } from "node:path";

import { Keypair } from "@solana/web3.js";

function readJSONSync(pathToFile: string) {

  const pathToJSON = resolve(pathToFile);

  const buffer = readFileSync(pathToJSON, "utf8");

  return buffer;

}

function readAsUInt8(bufferString: string) {

  return Uint8Array.from(JSON.parse(bufferString));

}

export function readKeypairFromFile(pathToFile: string) {

  return readAsUInt8(readJSONSync(pathToFile));

}
```

```
}  
  
export default function fromSecretKey(path: string) {  
    return Keypair.fromSecretKey(readKeypairFromFile(path));  
}
```

Currently, a sample keypair file was found to be located in `solana-tools-client/packages/cli/.keys/sLPjiiFxD2nvSiGZrb9bNDxEYPqLPwtCotk92699eEg.json`

Impact

Another user sharing the machine, or in case of a machine compromise, other parties may get access to the unprotected keypair, allowing it to take over the legitimate user's identity and perform actions (e.g. sign transactions) on behalf of that user.

Recommendation

While there's no possibility to completely avoid storing the keypair on the filesystem, encrypting it could be an option to improve security - as with wallet software, a password should be implemented and used to protect the keypair - encrypt for storage and decrypt while reading during runtime.

Note

The AgriDex team understands and recognizes the concern regarding the unencrypted key. Their approach to resolving the finding is:

1. Issue the token and mint the entire supply.
2. Remove the mint and freeze authorities.
3. Transfer the entire supply to the Treasury, controlled by a multi-sig wallet.

Status

Resolved

Low

[L-01] Solana CLI tools - X-Magic-Secret-Key is stored in the command line history, making it vulnerable to leaks

Description

The project allows for wallet creation which requires providing a Magic Link API Key. This key is a secret which allows for magic.link authentication and should be protected.

As per the below screenshot, the API key can be supplied as a command line argument:

```
### MagicLink

$ solana-tools wallet create-bulk \
  --provider magic-link \
  --api-key $MAGIC_LINK_API_KEY \
  ./wallet-list.csv
Job=<job_id>

Arguments:
  source_path          [Path] Path to source-file with a list of wallets

Options:
  --api-key [string]    Public API key
  --provider [magic-link] Provider for wallet creation
  -p, --payer [Path]   Path to custom keypair (default: active wallet)
  -h, --help            display help for command
```

Due to this, the API key will be stored e.g. in bash_history or similar command line logging utilities which are enabled by default on most operating systems. The risk related to it is that when there's a shared machine in use or a machine is compromised, it might be recovered from that place.

Recommendation

Load the key encrypted from a file or implement a secure password prompt similar to this:

<https://stackoverflow.com/questions/14591347/how-to-disable-stdin-echo-in-a-dart-console-application-for-password-input>

Status

Resolved



QA

[Q-01] Solana CLI tools - Potentially vulnerable dependency

Description

The tool npm audit detected one of the dependencies potentially vulnerable to a ReDoS issue. However, even if exploited, it is unlikely to have a significant impact on a command line tool, thus this is classified as a QA finding.

The below listing shows the output of the npm audit command in solana-tools-client directory:

```
npm audit
# npm audit report

micromatch <4.0.8
Severity: moderate
Regular Expression Denial of Service (ReDoS) in micromatch -
https://github.com/advisories/GHSA-952p-6rrq-rcjv
fix available via `npm audit fix`
node_modules/micromatch

1 moderate severity vulnerability

To address all issues, run:
  npm audit fix
```

Recommendation

Consider fixing the issue according to the command output.

Status

Resolved

[Q-02] Solana CLI tools - Certificate pinning can be used to improve security of external interactions

Description

The tool interacts with external services such as `magic.link` over secure SSL/TLS connection (HTTPS). Example is in `solana-tools-client/packages/core/src/wallet/index.ts`

By default, typescript verifies if the connection is secure, which is correct and should be considered sufficient to ensure user protection.

However, an extra step can be taken to ensure the solution's security: the certificate pinning mechanism, often used in Mobile applications. For services such as `magic.link`, for which the SSL certificate is known, it may ensure that if a DNS-based attack is applied, e.g., BGP hijacking, DNS rebinding, etc., the exact fingerprint of the known certificate is checked, and if it differs, rejected.

For example, Snyk has a guide on how to implement certificate pinning:

<https://snyk.io/blog/ssl-tls-pinning-node-js/>

Recommendation

Consider implementing certificate pinning for external SSL/TLS interactions.

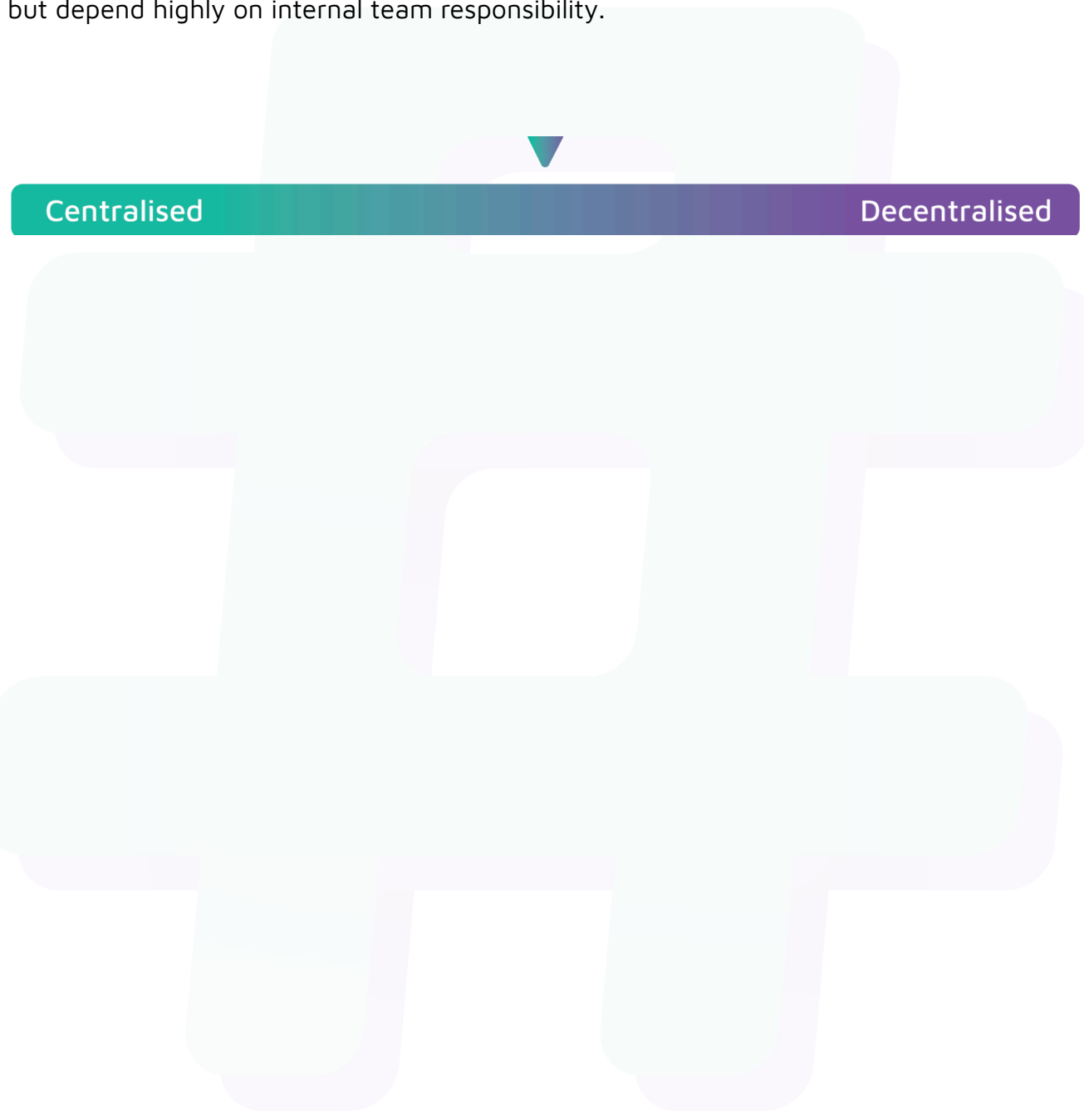
Status

Resolved

Centralisation

The Agridex project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlocks analysis, the Agridex project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our Penetration Tests a collaborative effort. The objective of our security Penetration Tests is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security Penetration Test process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future Penetration Tests. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the code base under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other Penetration Test results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the code bases in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the code base source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the Penetration Test makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this code base.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Code is deployed and executed on a platform. The platform, its programming language, and other software related to the code base can have their own vulnerabilities that can lead to attacks. Thus, the Penetration Test can't guarantee the explicit security of the Penetration Tested code bases.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.